



Industriële Besturingen programmeren

Week 10 – Tekstuele programmeertalen

Frank Goedhart

P2 - 2025/2026

DE HAAGSE
HOGESCHOOL

Vorige week

Netwerken

Ladderdiagrammen (LAD)

Function Block Diagram (FBD)

Timers en Counters

S7-GRAPH

Symbol table



Zelfstudie

Huiswerk!

- Bekijk: Kennisclip over de *Tag Table* (6:26)
- Maak: Zelfstudietoets (drie vragen)
- Lees: Siemens Manual – Tia Portal – PLC Tags
- Lees: 16.1.1 t/m 16.1.3 uit: Siemens Manual – Tia Portal - SCL

Vragen?



Vandaag

STL (Statement List)
SCL (Structured Control Language)
Rekenkundige operaties

Startup OB
Shared Data Block
Parameters bij FC, FB en OB
TEMP variabelen



Statement List (STL)

Ladder en FBD zijn symbolische programmeertalen: met behulp van symbolen zoals contacten en poorten is de bedoelde functionaliteit te programmeren.

Naast deze twee is er nog een taal: Statement List. (STL)

STL is de assemblertaal voor de S7 PLC's.

Je bent vrijer in het gebruik van instructies: constructies zijn mogelijk die in ladder of FBD niet kunnen.

We geven hier slechts een introductie tot STL.

(STL) Verknoping

Een groep logische operaties noemt men een verknoping (netwerk).

Een verknoping loopt van de eerste afvraag-instructie tot eerste opslag-instructie.

Voorbeeld in STL:

```
A      I 0.0      // vraag I0.0 af
A      I 0.1      // vraag I0.1 af, AND met I0.0
=      Q 1.1      // schrijf weg in Q1.1
```

(STL) Logische constructies

Er zijn ook logische constructies te maken zoals AND-OR. De AND gaat voor de OR.

Voorbeeld in STL:

```
A      M 0.0 // vraag M0.0 af
A      M 0.1 // vraag M0.1 af, AND met M0.0
O                               // tussenopslag OR
A      M 0.2 // vraag M0.2 af
A      M 0.3 // vraag M0.3 af, AND met M0.2
=      M 1.1 // schrijf weg in M1.1
```


(SCL) Structured Control Language

SCL is de implementatie van een hogere programmeertaal voor het programmeren van Siemens PLC's.

SCL is gebaseerd op Pascal en lijkt erg op de IEC 61131-3 implementatie van ST (Structured Text)

SCL wordt voornamelijk gebruikt voor:

- Wiskundige operaties
- String operaties
- Array operaties
- Communicatie
- Programmadoorloop

(SCL) Algemene regels

Een instructie kan meerdere regels lang zijn.

Elke instructie eindigt met een puntkomma ; .

SCL is niet hoofdlettergevoelig.

(SCL) Algemene regels

Een instructie kan meerdere regels lang zijn.

Elke instructie eindigt met een puntkomma ; .

SCL is niet hoofdlettergevoelig.

(SCL) Programmaverloop

```
1 // Program control with For Loop
2
3 □FOR counter := start_count TO end_count DO
4     // Statement section FOR
5     ;
6 □END_FOR;
```

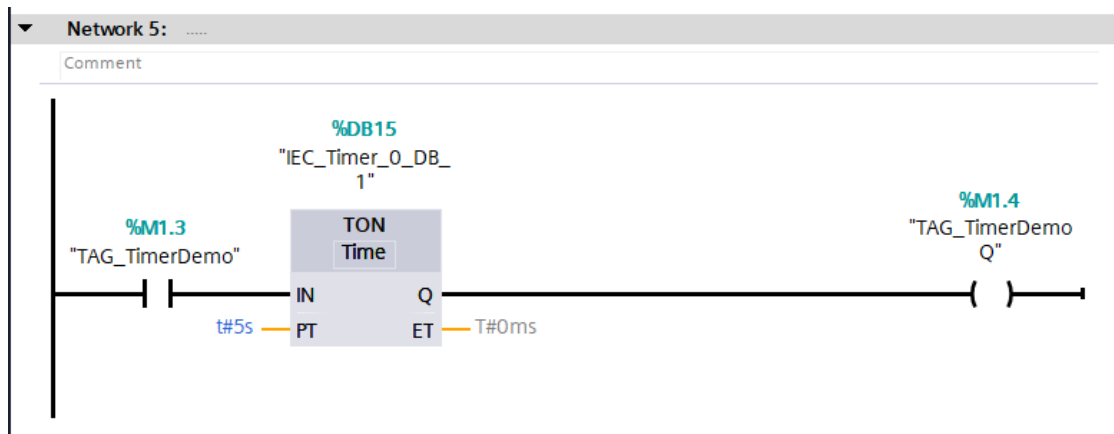
```
1 // Program control with While Loop
2
3 □WHILE condition DO
4     // Statement section WHILE
5     ;
6 □END_WHILE;
```

```
1 // Program control with Repeat Loop
2
3 □REPEAT
4     // Statement section REPEAT
5     ;
6 □UNTIL condition END_REPEAT;
```

```
1 // Program control IF
2 □IF condition THEN
3     // Statement section IF
4     ;
5 ELSE
6     // Statement section ELSE
7     ;
8 □END_IF;
```

(SCL) Timers aanroepen (Demo)

```
144 //Start een timer als de vacuum of laatlos vlag hoog is (deze wachten op een reset)
145 □ "IEC_Timer_0_DB".TON(IN:="flags".flags["FLAG_NO_VACUUM"] OR "flags".flags["FLAG_NO_LAATLOS"],
146     PT:=t#1000ms,
147     Q=>#resetFlag);
148
149 //Zet de vlaggen op 0 als de timer is verlopen.
150 □ IF #resetFlag THEN
151     "flags".flags["FLAG_NO_VACUUM"] := 0;
152     "flags".flags["FLAG_NO_LAATLOS"] := 0;
153 END_IF;
154
155
```



Rekenkundige operaties

Naast logische operaties kan een PLC ook rekenen.

Gerekend kan worden met de volgende typen op een Siemens PLC:

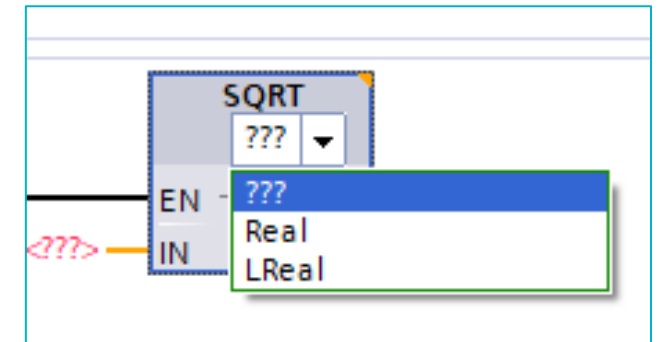
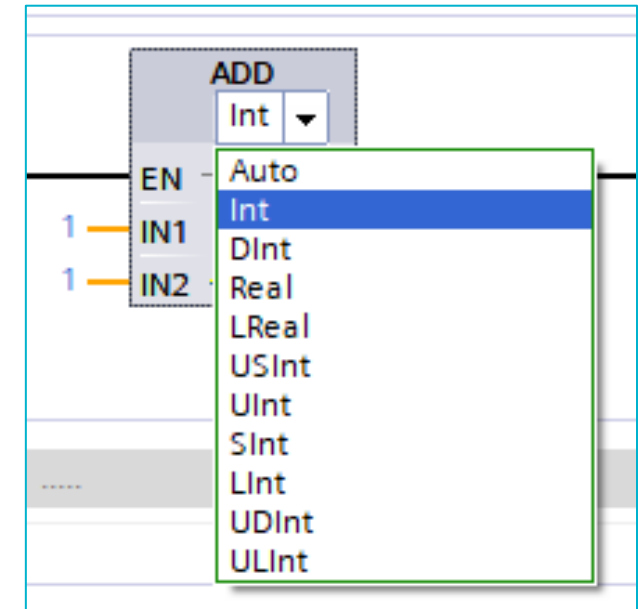
- Integer (16 bits)
- Double Integer (32 bits)
- Real (32 bits)

Rekenkundige operaties voor **int**:

- add, sub, mul, div, mod

Rekenkundige operaties voor **real**:

- add, sub, mul, div, abs, sqrt, sqr, ln, exp, sin, cos, tan, asin, acos, atan



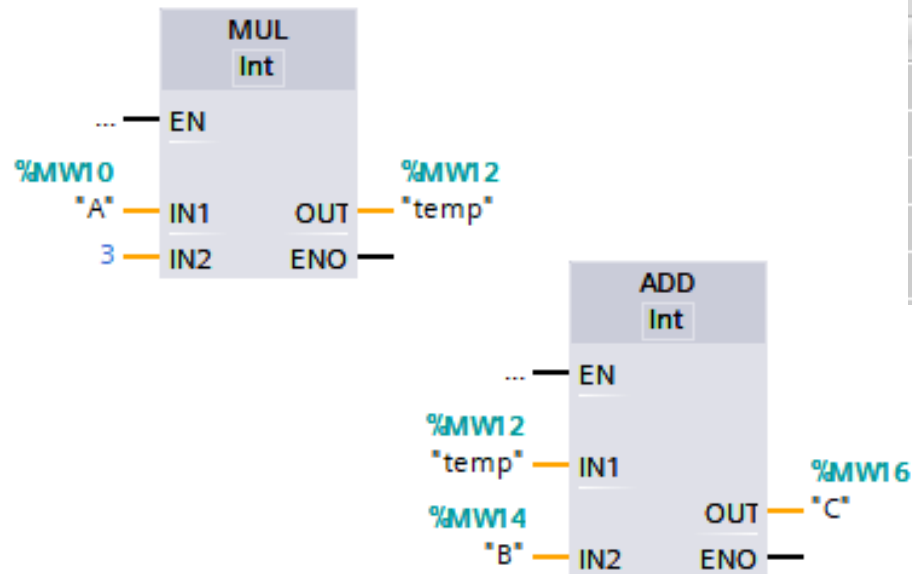
Voorbeeld rekenkundige operatie (1)

Losse bewerkingen

Als voorbeeld de uitwerking van de eenvoudige functie:

$$C = 3 * A + B$$

Let op: het datatype is integer maar het staat in een memory word.



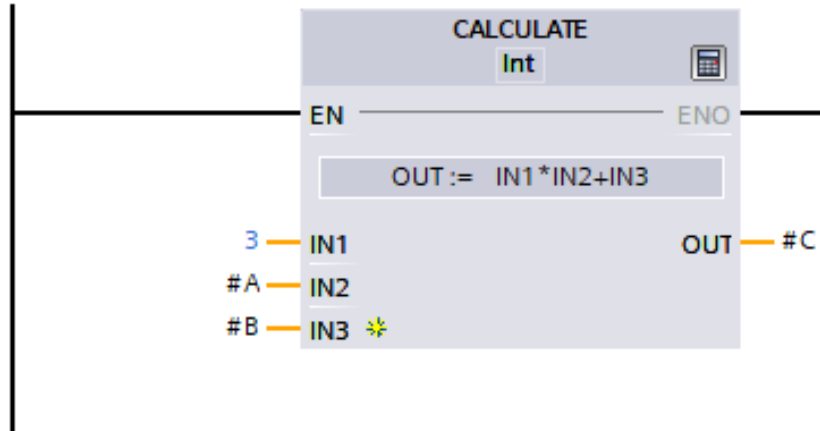
PLC tags					
		Name	Tag table	Data type	Address
65		A	Default tag table	Int	%MW10
66		temp	Default tag table	Int	%MW12
67		B	Default tag table	Int	%MW14
68		C	Default tag table	Int	%MW16
69		<Add new>			

Voorbeeld rekenkundige operatie (2)

Calculate

Als voorbeeld de uitwerking van de eenvoudige functie:

$$C = 3 * A + B$$

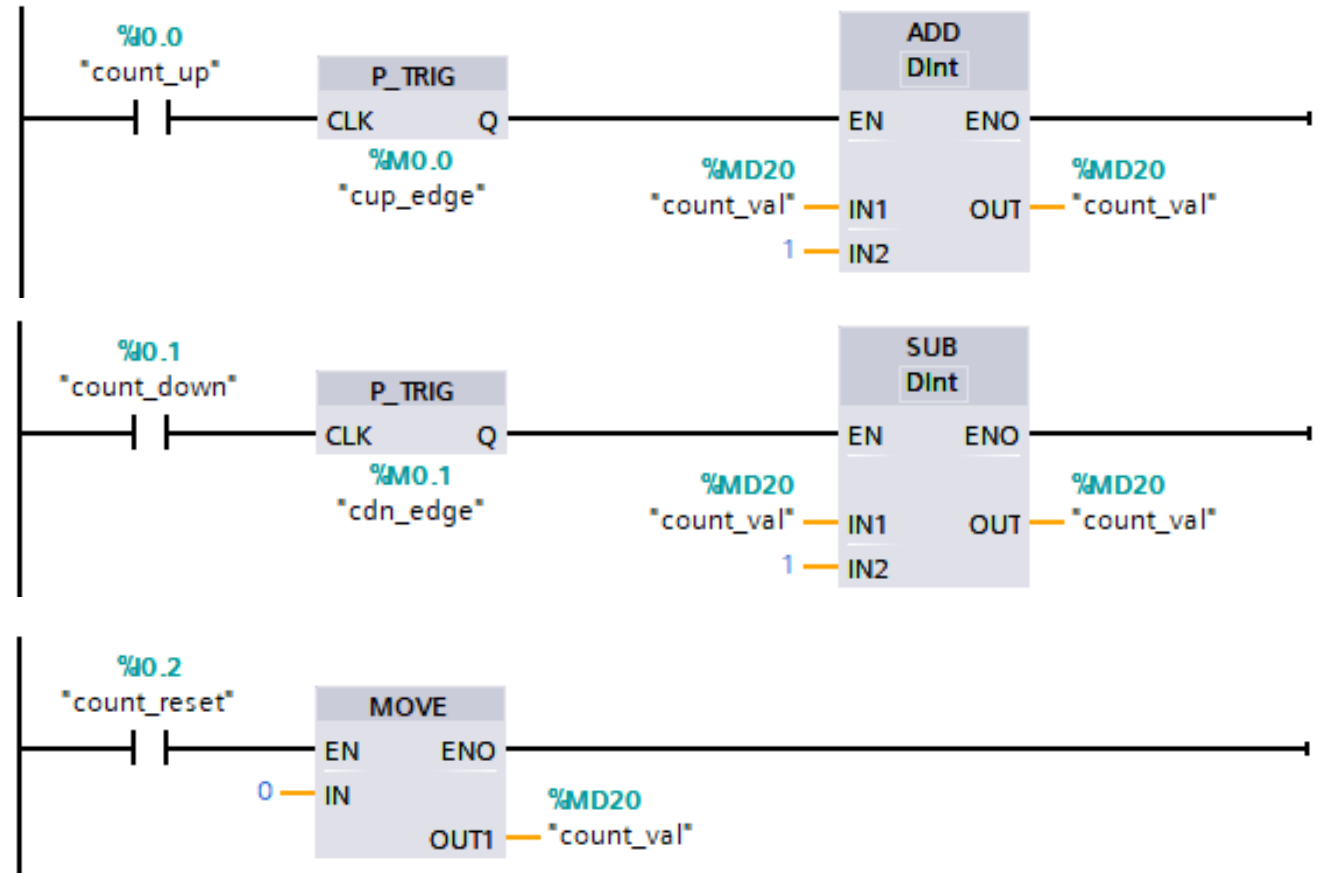


The screenshot shows a dialog box titled "Edit 'Calculate' instruction". It contains a text field for the formula, which currently shows "OUT := IN1*IN2+IN3". Below this, there is an "Example:" section with a text field containing the formula "(IN1 + IN2) * (IN1 - IN2)". Underneath, there is a "Possible instructions for Int:" section with a text field containing the list "+, -, *, /, Mod, Abs, Neg, Not / Inv". At the bottom right, there are two buttons: "OK" (highlighted in green) and "Cancel".

Eigen teller maken

Het is eenvoudig een eigen teller te maken die grote waarden kan bijhouden én negatieve waarden kan aangeven.

Let op: reset overheerst.



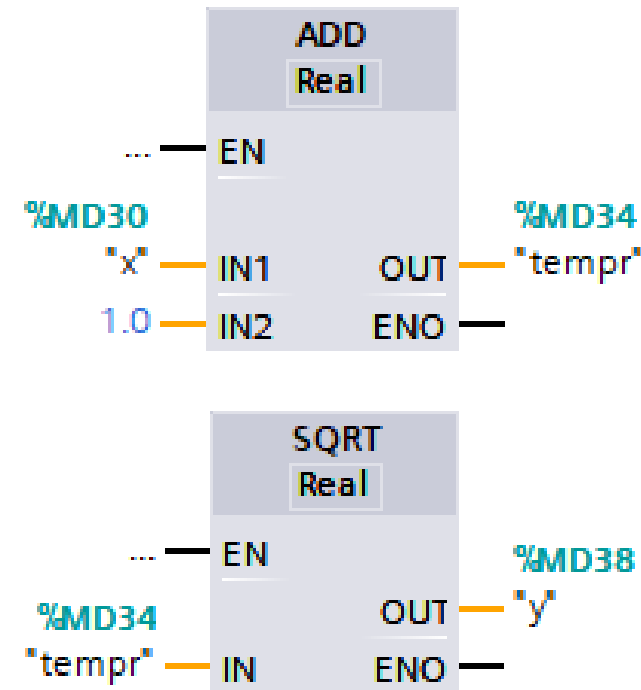
Voorbeeld rekenkundige operatie

Als voorbeeld hier de uitwerking van de functie:

$$y = \sqrt{x + 1}$$

Wat gebeurt er nu als $x < -1$ is?

```
14  "y" := Sqrt_REAL("x" / 1);
```



Voorbeeld rekenkundige operaties in SCL

```
1 // één plus één
2 "uitkomst" := 1 + 1;
3
4 // één min één
5 "uitkomst" := 1 - 1;
6
7 // één maal één
8 "uitkomst" := 1 * 1;
9
10 // één gedeeld door één
11 "uitkomst" := 1 / 1;
12
13 // etc...
```

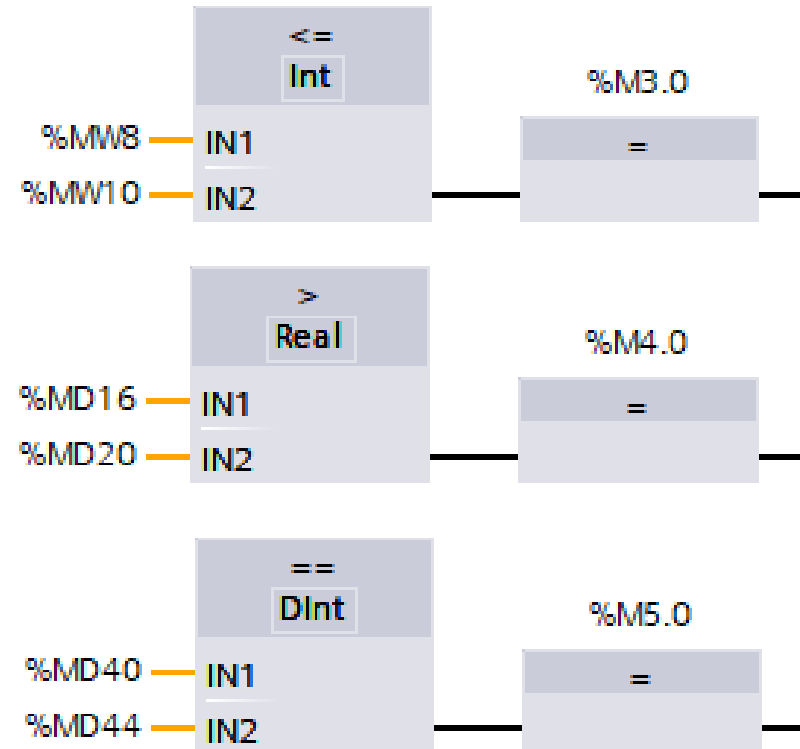
Math functions	
 ABS	Form absolute value
 MIN	Get minimum
 MAX	Get maximum
 LIMIT	Set limit value
 SQR	Form square
 SQRT	Form square root
 LN	Form natural logarithm
 EXP	Form exponential value
 SIN	Form sine value
 COS	Form cosine value
 TAN	Form tangent value
 ASIN	Form arcsine value
 ACOS	Form arccosine value
 ATAN	Form arctangent value

Vergelijken

De PLC kent ook een aantal vergelijkingen.

Variabelen van het zelfde type kunnen met elkaar vergeleken worden.

Praktisch elk type kan vergeleken worden.



Wanneer welke taal?

	LAD	FBD	GRAPH	SCL
Voordelen	Overzichtelijk bij simpele netwerken	Overzichtelijk bij simpele netwerken	Gemakkelijk stappen en sequenties implementeren	Overzichtelijk bij (wiskundige) bewerkingen
Beste toepassingen	Logische vergelijkingen (lijkt op relaistechniek)	Logische vergelijkingen (lijkt op digitale techniek)	Sequentiële programma-verloop	Bewerkingen en als volledige flexibiliteit nodig is
Zwaktes	Beperkte instructieset en onoverzichtelijk bij complexe netwerken	Onoverzichtelijk bij complexe netwerken	Beperkte flexibiliteit	Complex

Wanneer welke taal?

	LAD	FBD	GRAPH	SCL
Voordelen	Overzichtelijk bij simpele netwerken	Overzichtelijk bij simpele netwerken	Gemakkelijk stappen en sequenties implementeren	Overzichtelijk bij (wiskundige) bewerkingen
Beste toepassingen	Logische vergelijkingen (lijkt op relaistechniek)	Logische vergelijkingen (lijkt op digitale techniek)	Sequentiële programma-verloop	Bewerkingen en als volledige flexibiliteit nodig is
Zwaktes	Beperkte instructieset en onoverzichtelijk bij complexe netwerken	Onoverzichtelijk bij complexe netwerken	Beperkte flexibiliteit	Complex

Vandaag

STL (Statement List)
SCL (Structured Control Language)
Rekenkundige operaties

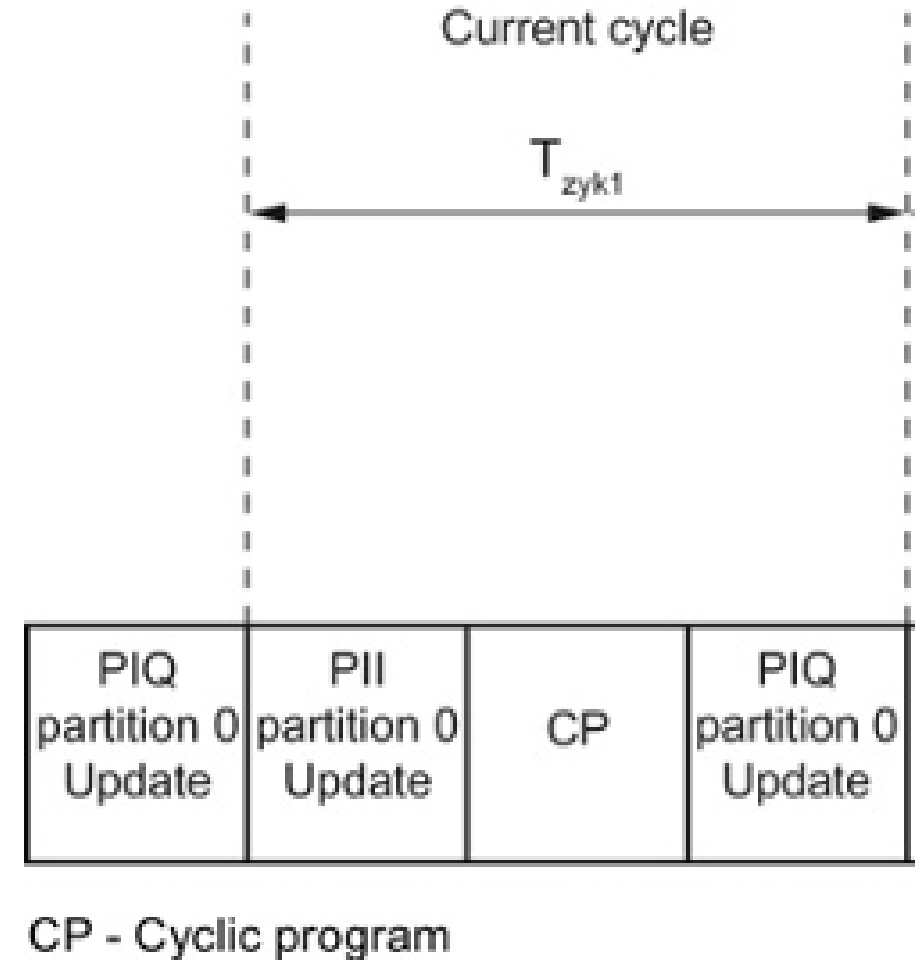
Startup OB
Shared Data Block
Parameters bij FC, FB en OB
TEMP variabelen



Startup en OB100

OB100 kan worden aangeroepen na elke (her)start van de PLC

OB100 wordt voor twee doeleinden gebruikt:
Initialisatie & Diagnose



Startup en OB100

slide_voorbeelden ▶ PLC_1 [CPU 315-2 PN/DP] ▶ Program blocks ▶ COMPLETE RESTART [OB100]

COMPLETE RESTART

Name	Data type	Offset	Default value	Comment
------	-----------	--------	---------------	---------

CALL

▼ **Block title:** "Complete Restart"

Comment

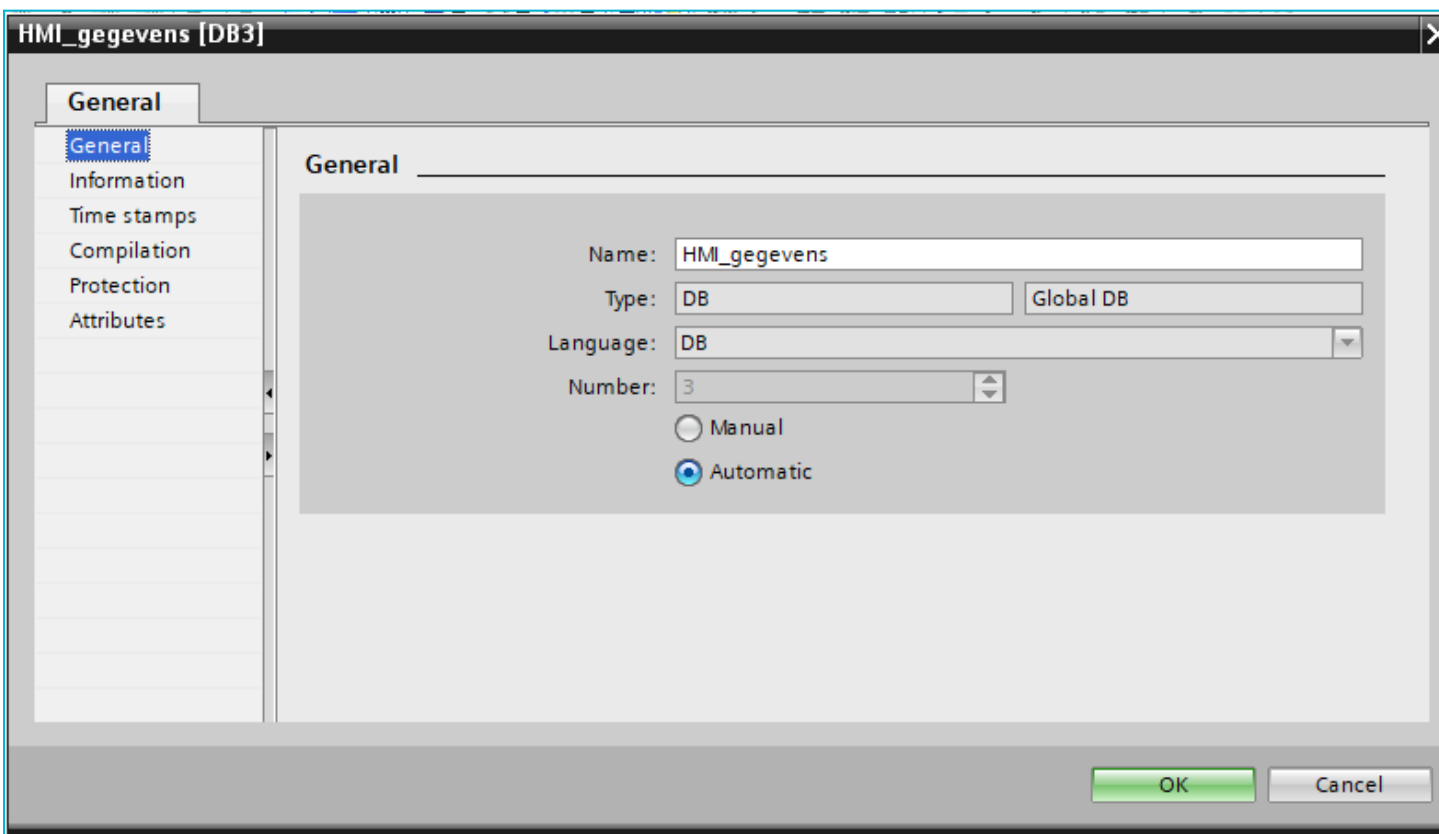
▼ **Network 1:** Reset variables

This network resets the start bit, sets the stop bit, and clears the counter value.

1	SET			
2	R	"start"		%M1.0
3	S	"stop"		%M1.1
4	L	0		0
5	T	"count_val"		%MD20

Global Data block

In een Shared/Global Data Block kan je gemeenschappelijke informatie opslaan en gebruiken.



Global Data block

In een Shared/Global Data Block kan je gemeenschappelijke informatie opslaan en gebruiken.

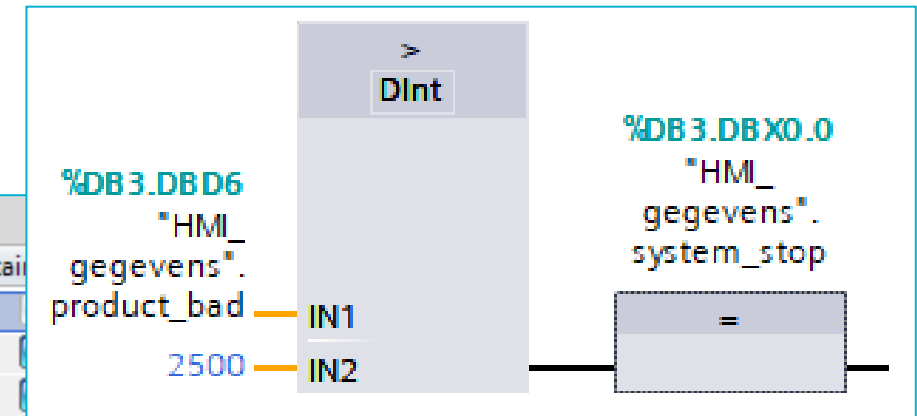
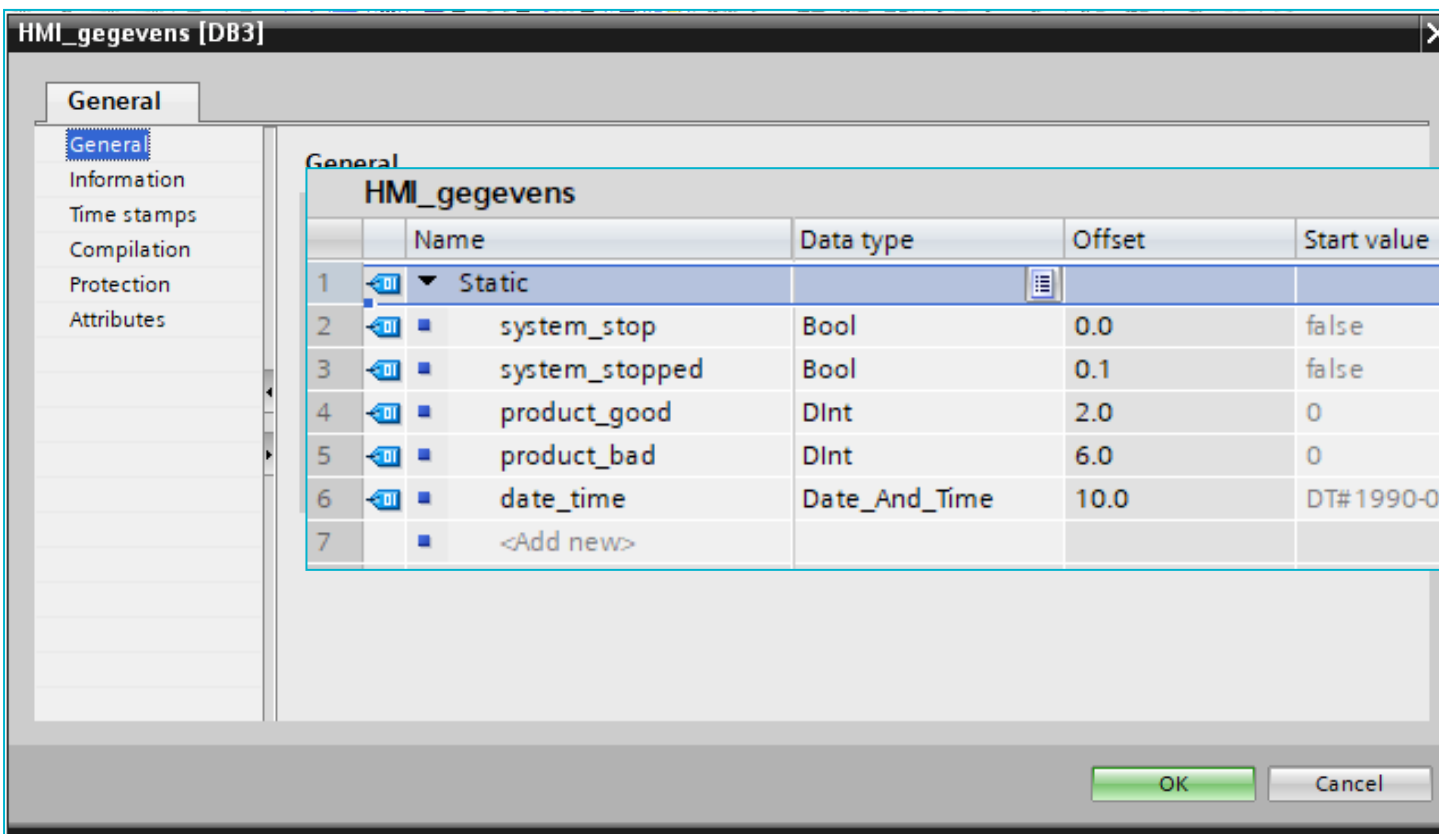
The screenshot shows a software window titled "HMI_gegevens [DB3]" with a close button (X) in the top right corner. On the left is a sidebar with a tree view containing the following items: "General" (selected), "Information", "Time stamps", "Compilation", "Protection", and "Attributes". The main area of the window is titled "General" and contains a table with the heading "HMI_gegevens". The table has the following columns: "Name", "Data type", "Offset", "Start value", "Retain", "Visible in HMI en...", and "Setpoint". The table contains seven rows of data:

	Name	Data type	Offset	Start value	Retain	Visible in HMI en...	Setpoint
1	Static				☐	☐	☐
2	system_stop	Bool	0.0	false	☒	☒	☐
3	system_stopped	Bool	0.1	false	☒	☒	☐
4	product_good	DInt	2.0	0	☒	☒	☐
5	product_bad	DInt	6.0	0	☒	☒	☐
6	date_time	Date_And_Time	10.0	DT# 1990-01-01-0	☒	☒	☐
7	<Add new>				☐	☐	☐

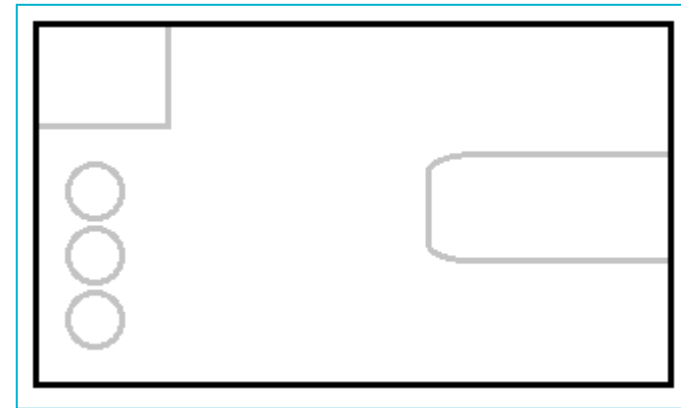
At the bottom of the window are two buttons: "OK" and "Cancel".

Global Data block

In een Shared/Global Data Block kan je gemeenschappelijke informatie opslaan en gebruiken.



Array's



In DB's is het mogelijk om array's op te slaan:

slide_voorbeelden ▶ PLC_1 [CPU 315-2 PN/DP] ▶ Program blocks ▶ ProductEigenschappen [DB4]

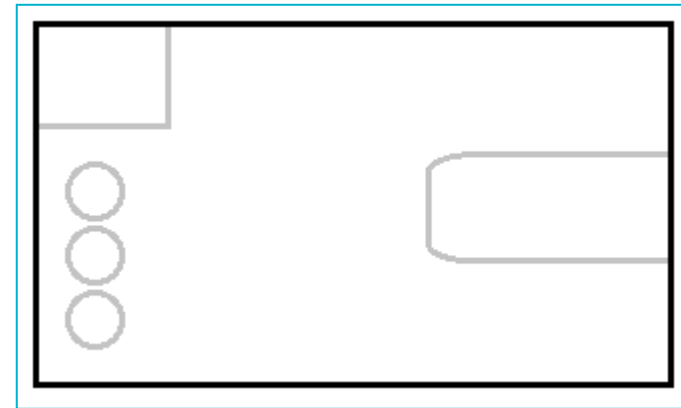
Keep actual values Snapshot Copy snapshots to start values Load start values as actual values

ProductEigenschappen

	Name	Data type	Offset	Start value	Retain	Visible in ...	Setpoint	Comment
1	Static				☐	☐	☐	
2	▶ plaatjes	Array[1..10] of Int	0.0		☒	☒	☐	
3	<Add new>				☐	☐	☐	

Array's

In DB's is het mogelijk om array's op te slaan:



slide_voorbeelden ▶ PLC_1 [CPU 315-2 PN/DP] ▶ Program blocks ▶ ProductEigenschappen [DB4]

Keep actual values Snapshot Copy snapshots to start values Load start values as actual values

	Name	Data type	Offset	Start value	Ret
1	Static				
2	▶ plaatjes	Array[1..10] of Int	0.0		
3	<Add new>				

ADD Int

... — EN

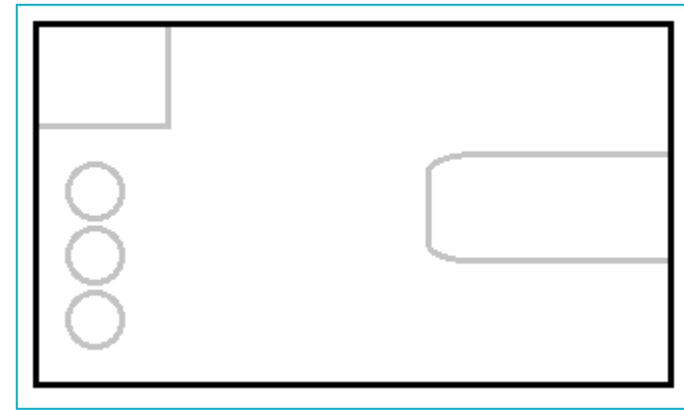
%DB4.DBW6
"ProductEigenschappen".
plaatjes[4] — IN1

1 — IN2

OUT — **%DB4.DBW6**
"ProductEigenschappen".
plaatjes[4]

ENO —

Structures

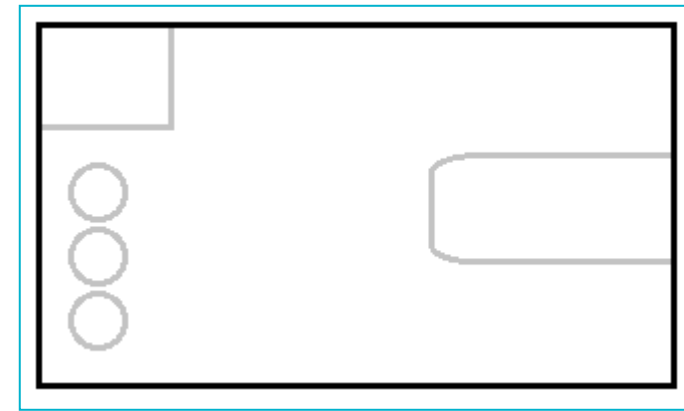


Het is mogelijk om structures te gebruiken. Hiermee worden bij elkaar horende gegevens gegroepeerd:

slide_voorbeelden ▶ PLC_1 [CPU 315-2 PN/DP] ▶ Program blocks ▶ GegevensPlaatje [DB5]								
GegevensPlaatje								
	Name	Data type	Offset	Start value	Retain	Visible in ...	Setpoint	Comment
1	Static				☐	☐	☐	
2	▼ kenmerken	Struct	0.0		☒	☒	☐	Kenmerken van plaatje
3	alleGaatjes	Bool	0.0	false	☒	☒	☐	
4	sleuf	Bool	0.1	false	☒	☒	☐	
5	inkeping	Bool	0.2	false	☒	☒	☐	
6	dik	Bool	0.3	false	☒	☒	☐	

Structures

Het is mogelijk om structures te gebruiken. Hiermee worden bij elkaar horende gegevens gegroepeerd:



slide_voorbeelden ▶ PLC_1 [CPU 315-2 PN/DP] ▶ Program block

Keep actual values Snapshot

GegevensPlaatje

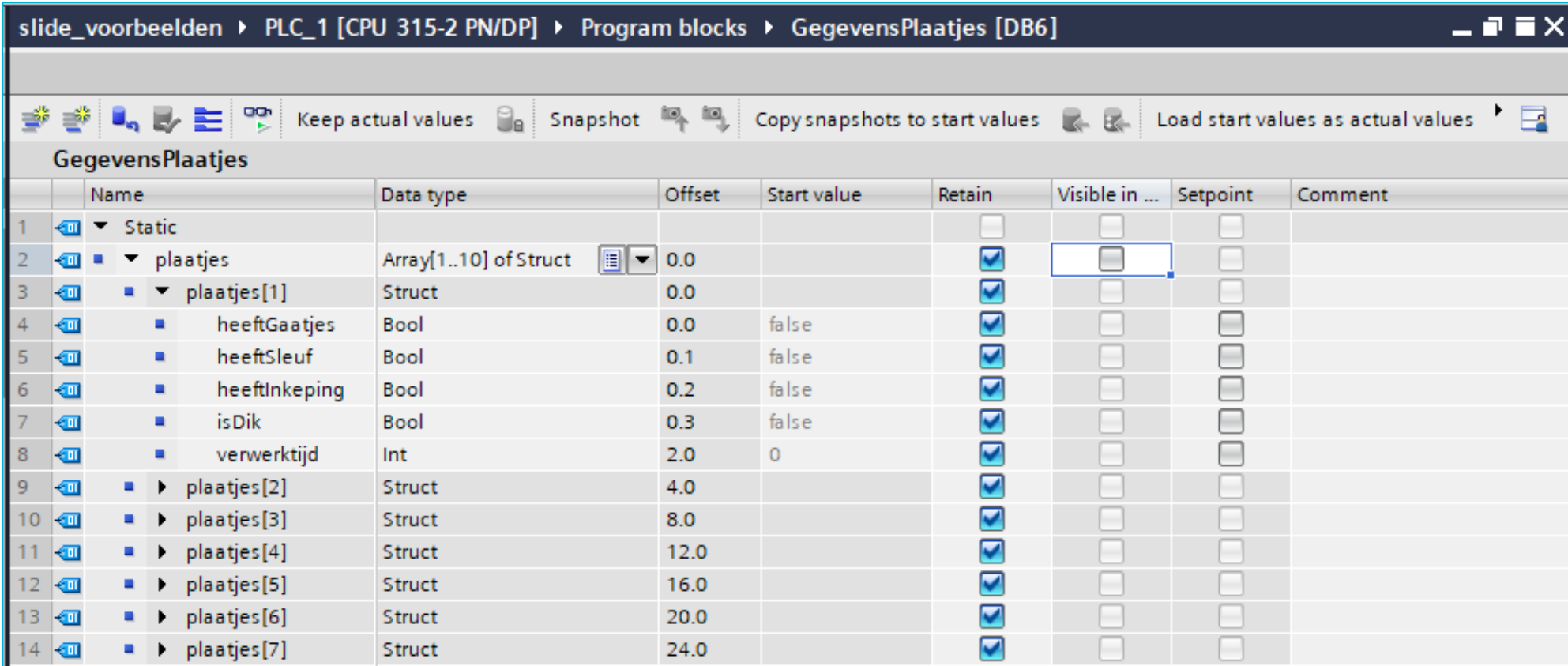
	Name	Data type	Offset	Start v
1	Static			
2	kenmerken	Struct	0.0	
3	alleGaatjes	Bool	0.0	false
4	sleuf	Bool	0.1	false
5	inkeping	Bool	0.2	false
6	dik	Bool	0.3	false

Diagram illustrating the use of structures in a PLC program:

- Input: `%DB5.DBX0.3` "GegevensPlaatje".kenmerken.dik
- Input: `%DB5.DBX0.2` "GegevensPlaatje".kenmerken.inkeping
- Operation: AND (&)
- Output: `%MO.1` "FreesDik"

Array van structures

Een array van structures is ook mogelijk in een DB:



slide_voorbeelden ▶ PLC_1 [CPU 315-2 PN/DP] ▶ Program blocks ▶ GegevensPlaatjes [DB6]								
Keep actual values Snapshot Copy snapshots to start values Load start values as actual values								
GegevensPlaatjes								
	Name	Data type	Offset	Start value	Retain	Visible in ...	Setpoint	Comment
1	▼ Static				☐	☐	☐	
2	▼ plaatjes	Array[1..10] of Struct	0.0		☒	☐	☐	
3	▼ plaatjes[1]	Struct	0.0		☒	☐	☐	
4	heeftGaetjes	Bool	0.0	false	☒	☐	☐	
5	heeftSleuf	Bool	0.1	false	☒	☐	☐	
6	heeftInkeping	Bool	0.2	false	☒	☐	☐	
7	isDik	Bool	0.3	false	☒	☐	☐	
8	verwerktijd	Int	2.0	0	☒	☐	☐	
9	▶ plaatjes[2]	Struct	4.0		☒	☐	☐	
10	▶ plaatjes[3]	Struct	8.0		☒	☐	☐	
11	▶ plaatjes[4]	Struct	12.0		☒	☐	☐	
12	▶ plaatjes[5]	Struct	16.0		☒	☐	☐	
13	▶ plaatjes[6]	Struct	20.0		☒	☐	☐	
14	▶ plaatjes[7]	Struct	24.0		☒	☐	☐	

Array van structures

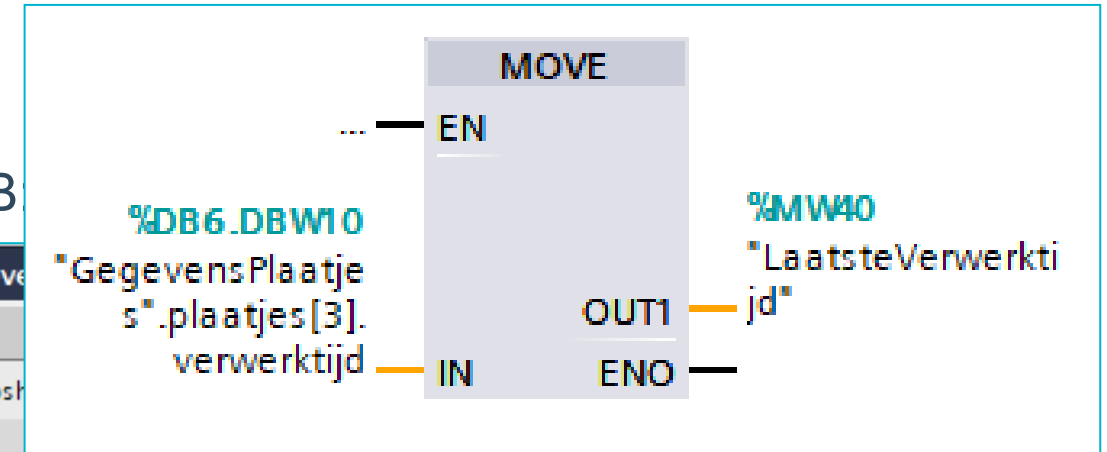
Een array van structures is ook mogelijk in een DB

slide_voorbeelden ▶ PLC_1 [CPU 315-2 PN/DP] ▶ Program blocks ▶ Gegevens

Keep actual values Snapshot Copy snapshot

GegevensPlaatjes

	Name	Data type	Offset	Start value	Retain	Visible in ...	Setpoint	Comment
1	Static				☐	☐	☐	
2	plaatjes	Array[1..10] of Struct	0.0		☒	☐	☐	
3	plaatjes[1]	Struct	0.0		☒	☐	☐	
4	heeftGaatjes	Bool	0.0	false	☒	☐	☐	
5	heeftSleuf	Bool	0.1	false	☒	☐	☐	
6	heeftInkeping	Bool	0.2	false	☒	☐	☐	
7	isDik	Bool	0.3	false	☒	☐	☐	
8	verwerktijd	Int	2.0	0	☒	☐	☐	
9	plaatjes[2]	Struct	4.0		☒	☐	☐	
10	plaatjes[3]	Struct	8.0		☒	☐	☐	
11	plaatjes[4]	Struct	12.0		☒	☐	☐	
12	plaatjes[5]	Struct	16.0		☒	☐	☐	
13	plaatjes[6]	Struct	20.0		☒	☐	☐	
14	plaatjes[7]	Struct	24.0		☒	☐	☐	



Parameters en variabelen FC & FB



In een *netjes geprogrammeerd* project worden **alle** variabelen via de interface van functies en functieblokken doorgegeven.

Parameters en variabelen FC & FB



Interface FC

IN	= ingangsparemeter (read-only)
OUT	= uitgangsparemeter (write-only)
IN_OUT	= in- en uitgangs-paremeter (read-write)
TEMP	= tijdelijke opslag
RETURN	= uitgangsparemeter (volgens norm)

Parameters moeten bij de bouwsteenoproep voorzien zijn van een adres (*pass by reference*).

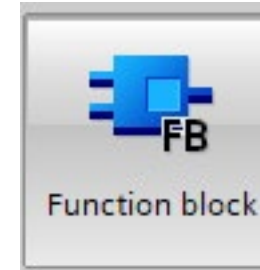
Parameters en variabelen FC & FB



Interface FC

IN	= ingangsparemeter (read-only)
OUT	= uitgangsparemeter (write-only)
IN_OUT	= in- en uitgangs-paremeter (read-write)
TEMP	= tijdelijke opslag
RETURN	= uitgangsparemeter (volgens norm)

Parameters moeten bij de bouwsteenoproep voorzien zijn van een adres (*pass by reference*).



Interface FB

IN	= ingangsparemeter (read-only)
OUT	= uitgangsparemeter (write-only)
IN_OUT	= in- en uitgangs-paremeter (read-write)
STAT	= DB lokaal geheugen (gebufferd)
TEMP	= tijdelijke opslag

Parameters mogen bij de bouwsteenoproep voorzien worden van een adres, maar hoeft niet (*pass by value*).

Parameteroverdracht FC

Pass by reference

slide_voorbeelden ▶ PLC_1 [CPU 315-2 PN/DP] ▶ Program block

MotorStarter

	Name	Data type	Offset
1	▼ Input		
2	motor1_start	Bool	
3	motor1_stop	Bool	
4	<Add new>		
5	▼ Output		
6	<Add new>		
7	▼ InOut		
8	<Add new>		
9	▼ Temp		
10	xyz	Bool	0.0
11	<Add new>		
12	▼ Constant		
13	<Add new>		
14	▼ Return		
15	MotorStarter	Bool	

Parameteroverdracht FC

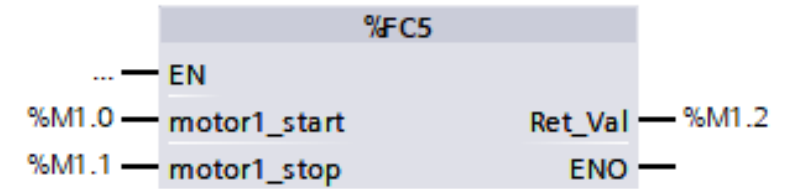
Pass by reference

slide_voorbeelden ▶ PLC_1 [CPU 315-2 PN/DP] ▶ Program block

MotorStarter

	Name	Data type	Offset
1	▼ Input		
2	motor1_start	Bool	
3	motor1_stop	Bool	
4	<Add new>		
5	▼ Output		
6	<Add new>		
7	▼ InOut		
8	<Add new>		
9	▼ Temp		
10	xyz	Bool	0.0
11	<Add new>		
12	▼ Constant		
13	<Add new>		
14	▼ Return		
15	MotorStarter	Bool	

OB1



Parameteroverdracht FC

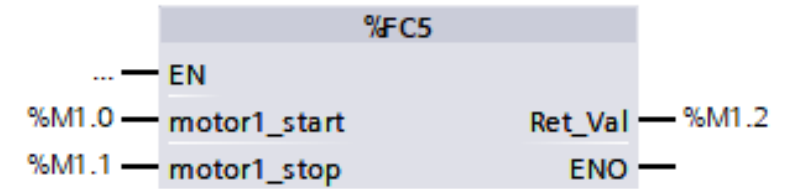
Pass by reference

slide_voorbeelden ▶ PLC_1 [CPU 315-2 PN/DP] ▶ Program block

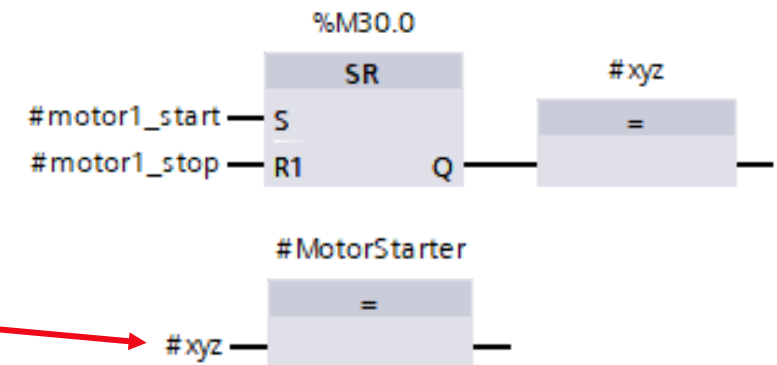
MotorStarter

	Name	Data type	Offset
1	▼ Input		
2	motor1_start	Bool	
3	motor1_stop	Bool	
4	<Add new>		
5	▼ Output		
6	<Add new>		
7	▼ InOut		
8	<Add new>		
9	▼ Temp		
10	xyz	Bool	0.0
11	<Add new>		
12	▼ Constant		
13	<Add new>		
14	▼ Return		
15	MotorStarter	Bool	

OB1



FC5



Parameteroverdracht FB (1)

Pass by value

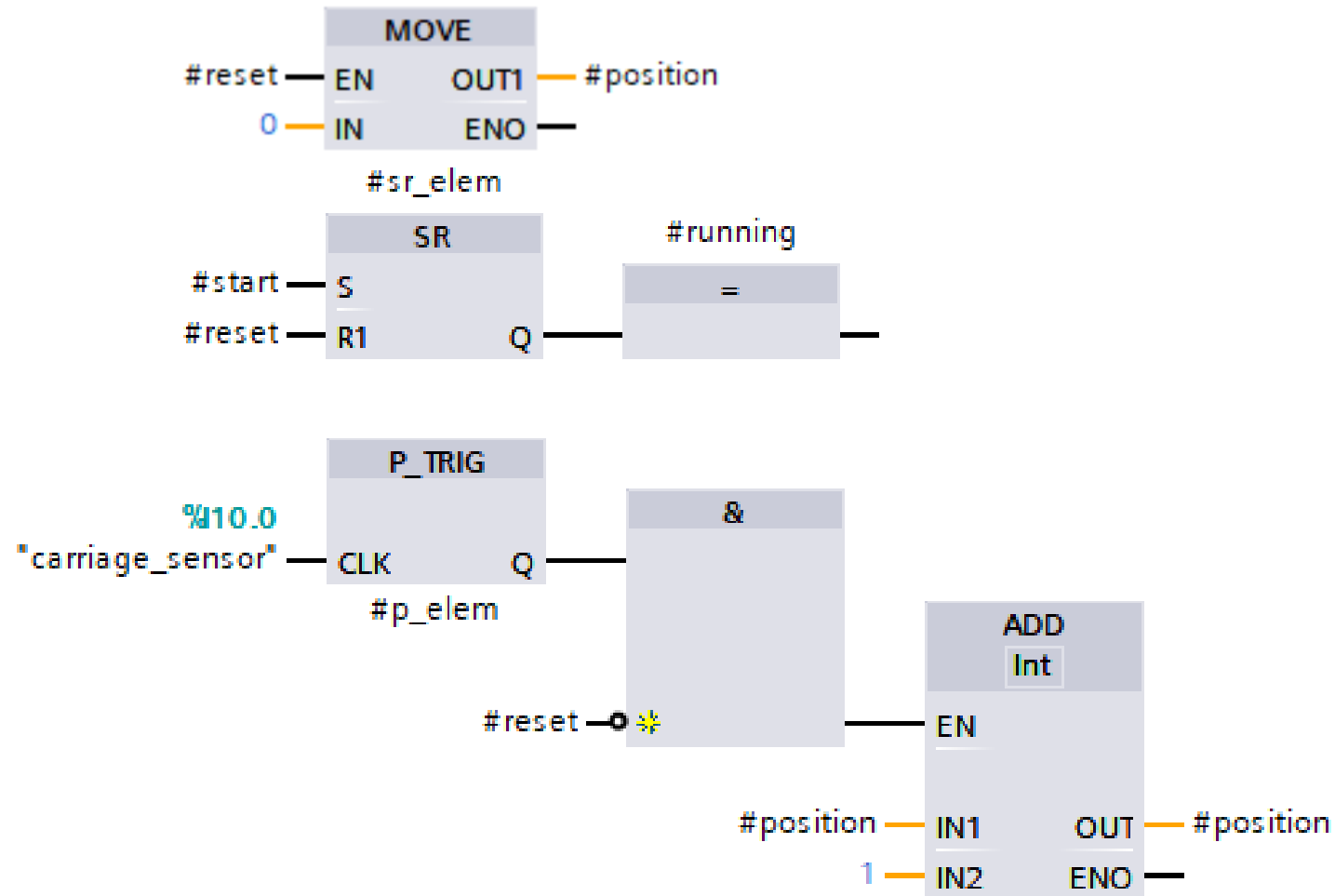
Parameters bij een FB worden doorgegeven via 'pass by value'. (waardes bij het aanroepende netwerk worden gekopieerd naar interne variabelen) Dit in tegenstelling tot bij een FC!

Inhoud van STAT (static) variabelen blijft behouden en is dus weer beschikbaar in volgende scancycle.

carriage_controller					
		Name	Data type	Offset	Default v..
1		▼ Input			
2		■ reset	Bool	0.0	false
3		■ start	Bool	0.1	false
4		▼ Output			
5		■ running	Bool	2.0	false
6		▼ InOut			
7		■ <Add new>			
8		▼ Static			
9		■ position	Int	4.0	0
10		■ sr_elem	Bool	6.0	false
11		■ p_elem	Bool	6.1	false
12		■ <Add new>			
13		▼ Temp			
14		■ <Add new>			
15		▼ Constant			
16		■ <Add new>			

Parameteroverdracht FB (2)

Pass by value

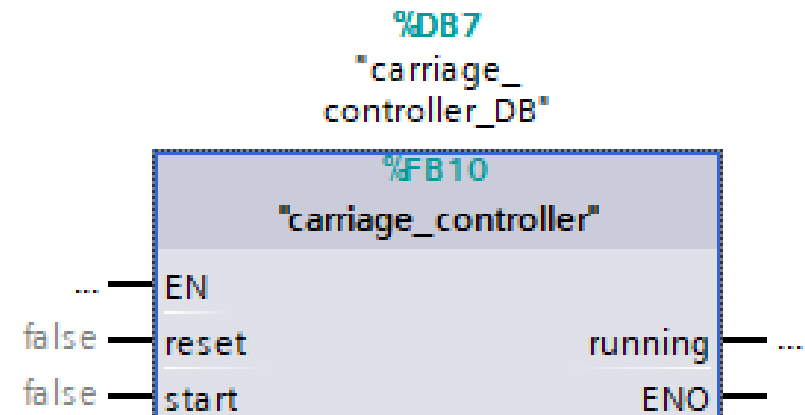
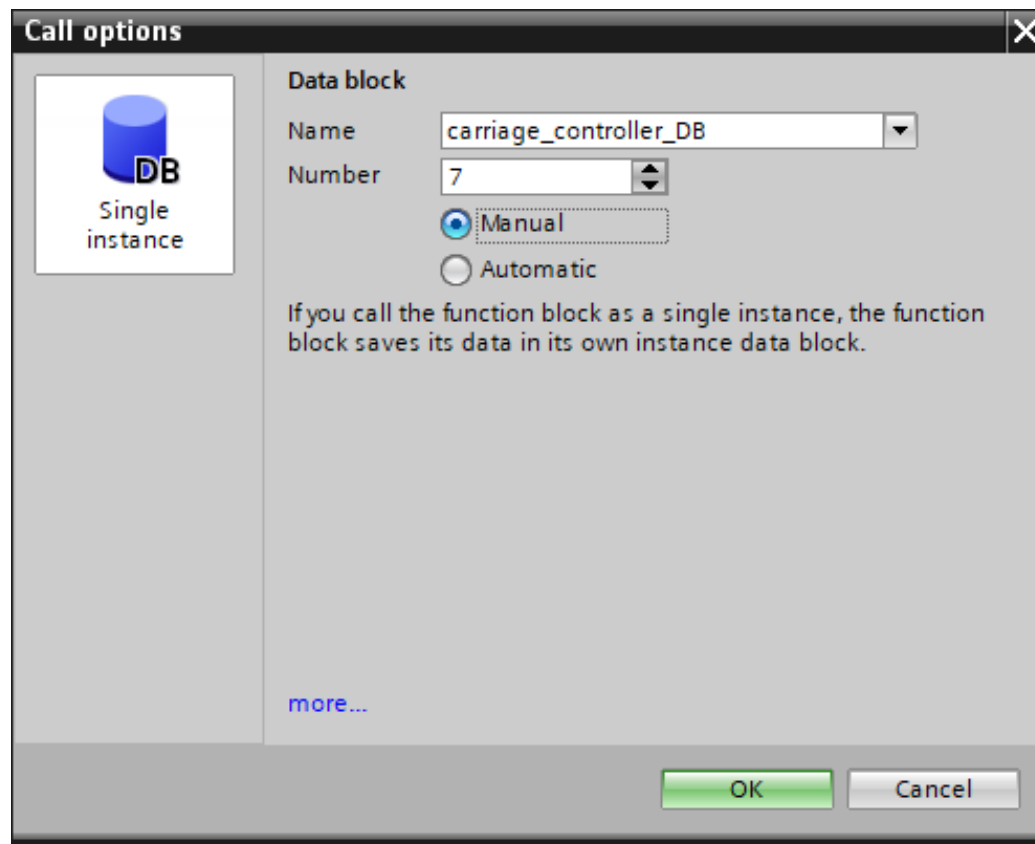


carriage_controller		
		Name
1	DI	Input
2	DI	reset
3	DI	start
4	DI	Output
5	DI	running
6	DI	InOut
7		<Add new>
8	DI	Static
9	DI	position
10	DI	sr_elem
11	DI	p_elem
12		<Add new>

Parameteroverdracht FB (3)

Pass by value

Bij het invoeren van een FB-aanroep wordt gevraagd of de bijhorende DB aangemaakt moet worden.



Parameteroverdracht FB (4)

Pass by value

De DB is dan als volgt ingevuld:

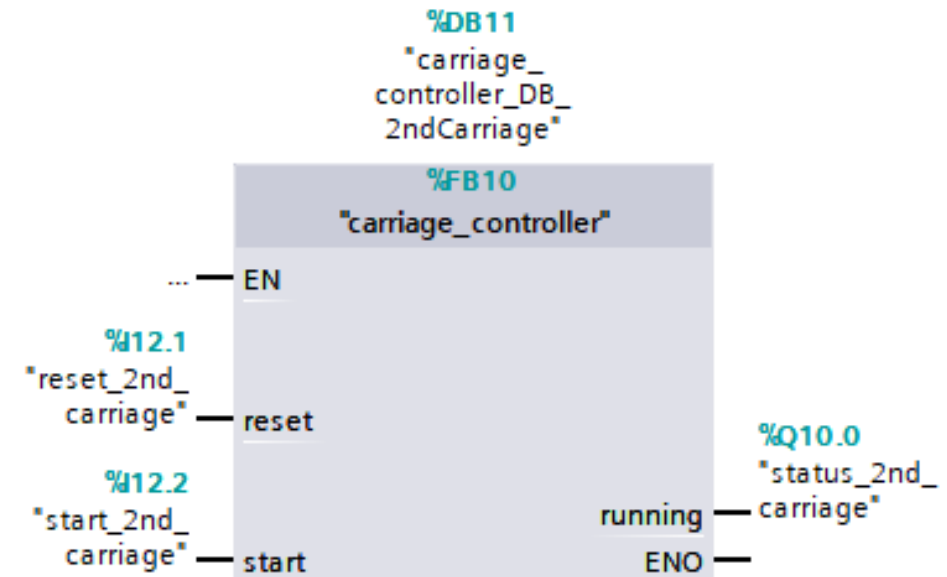
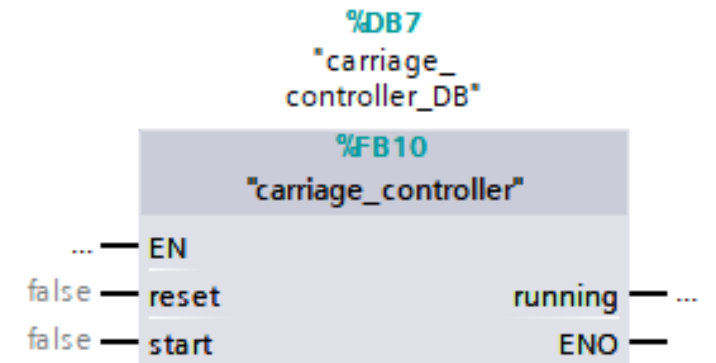
carriage_controller_DB					
	Name	Data type	Offset	Start value	Retain
1	▼ Input				☐
2	■ reset	Bool	0.0	false	☒
3	■ start	Bool	0.1	false	☒
4	▼ Output				☐
5	■ running	Bool	2.0	false	☒
6	InOut				☐
7	▼ Static				☐
8	■ position	Int	4.0	0	☒
9	■ sr_elem	Bool	6.0	false	☒
10	■ p_elem	Bool	6.1	false	☒

Parameteroverdracht FB (5)

Pass by value

Let op: Een instance DB kan slechts met één FB samenwerken (b.v. DB7 met FB10).

Een FB kan wel vaker aangeroepen worden en daarbij worden dan dus verschillende instance DB's gekoppeld.



Lokale stack

De lokale stack is een geheugengebied waar een OB, FB of FC gegevens kan aanpassen.

Bij een FC staan alleen TEMP-parameters op de lokale stack.

Bij een FB staan ook alleen TEMP-parameters in de lokale stack.

- Overige parameters staan in de Instance Data Block

Bij een OB staan OB-specifieke gegevens in de lokale stack.

Zelfstudie

Na de les ga je aan de slag met:

- Bekijk: Kennisclip over de Startup OB  en beantwoord de vraag in de video

Achtergrondinformatie is te vinden in Hoofdstuk 1 uit:

<https://support.industry.siemens.com/cs/document/45523446/simatic-step-7-v5-5-statement-list-%28stl%29-for-s7-300-and-s7-400-programming?dti=0&pnid=14341&lc=en-VW>

Verdere informatie over rekenkundige functies en vergelijkingen kan worden gevonden in:

<https://support.industry.siemens.com/cs/document/45522487/simatic-step7-v5-5-function-block-diagram-%28fbd%29-for-s7-300-and-s7-400-programming?dti=0&pnid=14342&lc=en-VW>



Vandaag

STL (Statement List)
SCL (Structured Control Language)
Rekenkundige operaties

Startup OB
Shared Data Block
Parameters bij FC, FB en OB
TEMP variabelen

